New for M3 — feature branch → develop → main

nautilus

Team 5 GitHub *workflow.*

From M3 on, all code goes feature branch → develop → main, every merge needs an approved pull request, and every review must comment on at least one review criterion. This follows the professor's "Branch and Code Review with GitHub" lecture.

CSC 648/848 · SECTION 6 · TEAM 5 · OCT 27, 2026 · PHILIP CHEN, GITHUB MASTER

01 **Branch**
Start from develop

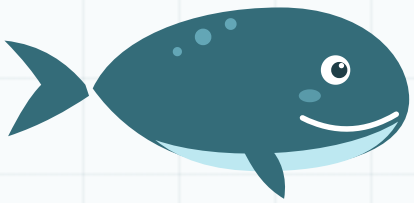
02 **Commit**
Small, clear messages

03 **Review**
One real approval

04 **Merge**
Into develop







M3 CHECKLIST

What the TA grades (M3 Appendix II)



After the M3 demo, the TA reviews our GitHub repo directly. Each check point maps to a section of this guide:

TA check point	Covered in
`Dev and feature branches are properly set up and used`	1. Branches, 2. Everyday workflow
`How many code reviews have been done since the M3 announcement`	4. Pull requests and reviewers
`How meaningful the reviews are ("looks good" doesn't count)`	5. Meaningful review
`Back-end and front-end frameworks deployed correctly`	Live site at csc648-team05.vercel.app (Marco), not covered by this guide

The TA can only see what's on GitHub. Reviews done verbally or in Discord don't count. Start opening and reviewing PRs now, since the count starts from the M3 announcement.



SECTION 01

Branches

01

Branch	What it's for	Created from	Merges into	Who can merge
`main`	Production-ready code. Vercel deploys the live site from here.	—	—	Only through a release PR, approved by Om
`develop`	Integration branch: everyone's finished, reviewed work. Must always run.	main (once)	main, through a release branch	Through an approved PR only
`feature/<name>-<task>`	One task or feature, e.g. feature/aksh-dashboard-ui	develop	develop	Author, after 1 approval
`bugfix/<name>-<issue>`	One bug fix, e.g. bugfix/marco-search-500	develop	develop	Author, after 1 approval
`release/m3 (m4, m5)`	Milestone release prep: final small fixes only, no new features	develop	main and back into develop	Philip, after Om approves

Rules: nobody pushes directly to main or develop. Feature branches are short-lived: merge or delete them within a few days. GitHub deletes the branch automatically once it merges.



Releasing a milestone (develop → main)

1. Freeze develop a few days before the deadline: small fixes only, no new features.
2. Philip creates `release/m3` from develop and pushes it.
3. Philip opens a PR with **base: main** and **compare: release/m3**, and requests Om (or Aksh as backup).
4. Om checks it is ready for the live site. Last fixes are committed to the release branch.
5. Philip merges into main and Vercel deploys the live site.
6. Any last fixes go back into develop with a small PR from the release branch.

SECTION 02

Everyday workflow

02

1. Start from the latest develop:

```
git checkout develop
git pull
git checkout -b feature/yourname-short-task
```

2. Work in small commits (see section 3). Push your branch at least once a day so others can see progress:

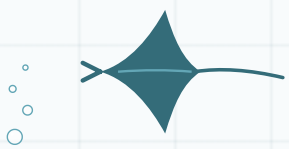
```
git add <files>
git commit
git push -u origin feature/yourname-short-task
```

3. Before opening a PR, bring in the newest develop and fix any conflicts:

```
git pull origin develop
```

4. Open a pull request on GitHub with **base: develop** (not main). Fill in the PR template.
5. Test your change on the Vercel preview link the PR gets, not only on localhost.
6. Wait for an approving review. Answer every comment, either by changing the code or replying.
7. Once approved and every comment is resolved, **the author merges**. GitHub lets anyone on the team click Merge, so leave it to the author in case they have a last fix. The branch deletes itself; to clean up your laptop, run `git checkout develop`, `git pull`, then `git branch -d <your-branch>`.

Never commit .env files, the Gemini API key, Supabase keys, passwords or demo account credentials.



SECTION 03

Commit messages

03

One commit = one focused change, ideally under 20–30 changed lines. Don't mix unrelated changes in one commit.

Format (from the lecture):

```
Add search box to PRD dashboard

Calls GET /api/prds?search= as the user types and shows
matching PRDs. Empty results show a "No PRDs match" message.

- Debounce input by 300 ms
- Reuse PRDRow component
```

- **Summary line:** 72 characters or less, imperative mood ("Add", "Fix", "Update" — not "Added" or "Fixes").
- **Blank line**, then a body explaining what and why, wrapped at 72 characters. Bullets are fine.

Bad	Good
`fixed stuff`	Fix 500 error when searching with empty text
`update`	Update README with M3 setup steps
`WIP / asdf`	Add Gemini client with timeout and retry
`changes`	Rename brief to intake_answer in backend schemas



SECTION 04

Pull requests and reviewers

04

The lecture recommends the backend and frontend leads as reviewers. Every PR into develop needs **1 approval** from someone other than the author.

Change touches	Primary reviewer	Backup reviewer
`app/frontend/`	Aksh	Om
`app/backend/ (API, database, schema)`	Marco / Vineela	Marco / Vineela
`AI / Gemini code and prompts`	Giovannie / Om	Giovannie / Om
`README, .github/ (GitHub setup)`	Philip	Leman / Aksh
`milestones/ (M1–M5 documents)`	Leman / Om	Philip
`Release PR (release/mX → main)`	Om	Philip / Aksh

AI PRs also notify Marco and Vineela, because AI code runs in the backend. They can comment if a change affects the API or database, but Giovannie or Om do the main review.

- If you're listed as a reviewer for your own change, another person in that row reviews it (for example, Vineela's backend PR is reviewed by Marco).
- Reviewers respond within 24 hours, sooner near a deadline.
- A PR should be small enough to review in about 15 minutes. Split big features into several PRs.
- PR description must say: what changed, why, how to test it, and the requirement ID (e.g. R_5.2).
- Do all review discussion in the PR on GitHub, not in Discord, so the TA can see it. If you talked it through in person, summarize the outcome as a PR comment.



SECTION 05

What counts as a meaningful review

05

"Looks good to me" alone is not a review. Before approving, the reviewer checks the criteria below and leaves at least one specific comment, even if it's a question or a small suggestion.

Criterion	Question to ask
Design	Does it fit our architecture (React → FastAPI → Supabase)? Could it be simpler?
Functionality	Does it do what the PR says? Did you try it on the preview link?
Complexity	Could another teammate understand and change this later?
Tests	Is it tested, or at least were the test steps in the PR followed?
Naming	Do names match Section 1 data definitions (e.g. intake_answer, prd_version)?
Comments	Are tricky parts explained?
Style	Does it match the existing code style?
Documentation	Did the README or M3 report need an update?
Secrets	No keys, passwords or .env files in the diff?

On GitHub, choose **Approve**, **Comment** or **Request changes**. Use Request changes for real problems (bugs, secrets, broken build), not style preferences.

